phase3_issuance_guard.sh

phase3_issuance_guard.sh

Revision: 1 **Last modified:** 2026-07-01T00:00:00Z

Standing §11.4.135 regression guard for the Let's Encrypt **Phase 3 hermetic DNS-01 issuance** (task #59). It drives the REAL issuance artifact `deploy/letsencrypt/phase3_hermetic_issue.sh` and proves that the hermetic ACME stack still obtains a **real** TLS certificate against a local Pebble — AND that the exact design-gap regression the CoreDNS authoritative-SOA front fixes (2026-07-01) stays fixed. certmagic's DNS-01 flow determines the DNS zone via an SOA walk **before** it presents the challenge TXT; `challtestsrv` answers `NOTIMP` to SOA (it has no authoritative mode), which blocked issuance ("could not determine zone … NOTIMP"). CoreDNS was inserted as an authoritative SOA front for `hermetic.test`, and Caddy's `ACME_RESOLVERS` was pointed at `coredns:53`. If a later change reverts `ACME_RESOLVERS` back to `challtestsrv:8053` (or drops CoreDNS), issuance silently breaks — this guard catches that regression.

Overview

Field	Value
Path	`tests/letsencrypt/phase3_issuance_guard.sh`
Under guard	`deploy/letsencrypt/phase3_hermetic_issue.sh` (issuance) · `deploy/letsencrypt/compose.hermetic.yml` (`ACME_RESOLVERS` default) · `deploy/letsencrypt/coredns/Corefile` (SOA front) · `Caddyfile`
Verdicts producer	`tests/letsencrypt/cert_analyzer.sh` (`cert_chain_roots_in` / `cert_san_matches`)
Kind	

Field	Value
	Live issuance guard — **boots the hermetic podman-compose stack via phase3** (rootless, §11.4.161); conductor-only (§11.4.119)
Anti-bluff anchors	§11.4.107, §11.4.108, §11.4.115, §11.4.135, §11.4.3, §11.4.50, §1.1
Exit codes	`0` = PASS, `1` = FAIL, `2` = SKIP (topology absent / precondition unmet, §11.4.3)

Prerequisites

- POSIX `sh` on `PATH` (to run the guard itself + parse the phase3 contract).
- To actually issue (conductor-side, GREEN or RED): `podman` + `podman-compose` (rootless, §11.4.161) and the built Caddy image `localhost/helix_proxy/caddy-challtestsrv:2.8.4` (produced by `deploy/letsencrypt/build.sh`). When these are absent the guard emits an honest §11.4.3 topology **SKIP** (exit 2), never a fake pass.
- No operator secret is required — the hermetic DNS-01 path uses the unauthenticated `challtestsrv` (§11.4.10). Pebble regenerates its issuance CA every boot; phase3 fetches THIS run's CA for the chain check.
- **Do NOT run this guard during background authoring** — it boots containers, which is conductor-only under §11.4.119 (single-resource-owner). The conductor runs it as part of the release-gate sweep (§11.4.40).

Usage examples

```
# GREEN standing guard — shipped defaults (ACME_RESOLVERS =>
        coredns:53).
# PASS iff phase3 issues a real cert AND
        cert_analyzer_verdicts.txt shows both
# cert_chain_roots_in: PASS and cert_san_matches: PASS.
tests/letsencrypt/phase3_issuance_guard.sh

# RED (reproduce) — broken resolver bypasses CoreDNS so the
        certmagic SOA walk
# hits challtestsrv NOTIMP => no cert. PASS iff phase3 FAILS.
RED_MODE=1 tests/letsencrypt/phase3_issuance_guard.sh
```

§11.4.115 RED_MODE polarity

Mode	Meaning	PASS condition
`RED_MODE=0` (default, GREEN standing guard)	Run phase3 with shipped compose defaults	phase3 exits `0` **and** the produced `cert_analyzer_verdicts.txt` contains `cert_chain_roots_in: PASS` **and** `cert_san_matches: PASS`
`RED_MODE=1` (reproduce)	Run phase3 with `ACME_RESOLVERS=challtestsrv:8053` (bypasses CoreDNS)	phase3 **FAILS** (exit non-`0`, non-`2`) — the guard catches the CoreDNS-SOA-front regression

A RED run in which phase3 still succeeds is a §11.4.7 finding (the regression is not caught) and the guard reports **FAIL**. Topology absent in either mode ⇒ **SKIP** (exit 2) — a reproduction cannot run without the stack.

RED injection mechanism (no `deploy/` change required)

`compose.hermetic.yml` declares `- ACME_RESOLVERS=${ACME_RESOLVERS:-coredns:53}` and `phase3_hermetic_issue.sh` **neither forces nor unsets** `ACME_RESOLVERS`. Exporting `ACME_RESOLVERS=challtestsrv:8053` into phase3's environment therefore propagates through **phase3 → podman-compose → the compose `${..:-}` default → Caddy**, pointing certmagic's zone-determination resolver directly at `challtestsrv` (which answers `NOTIMP` to SOA) and bypassing the CoreDNS authoritative front — a faithful reproduction of the exact pre-fix bug. In GREEN the guard `unset`s any inherited `ACME_RESOLVERS` in a subshell so the shipped `coredns:53` default applies deterministically (§11.4.50). CoreDNS still boots in RED; it is simply never queried by Caddy.

Contingent phase3 one-liner (documented, NOT applied here)

As of phase3 HEAD **2026-07-01, NO change is needed**. If a future phase3 hardening pins or unsets `ACME_RESOLVERS` (e.g. adds `unset ACME_RESOLVERS` or `export ACME_RESOLVERS=coredns:53`) it would defeat this RED injection. The conductor (who owns `deploy/`) should then, right after phase3's `export CADDY_IMAGE ...` line (~line 101), honour an externally-supplied value instead of forcing it:

```
export ACME_RESOLVERS="${ACME_RESOLVERS:-coredns:53}"
```

so the guard can still inject the broken resolver.

Suite wiring (conductor owns `tests/run-tests.sh`)

This guard mirrors the sibling §11.4.135 guards' `0`=PASS / `1`=FAIL convention, plus a third code **`2`=SKIP** (this guard boots containers and is conductor-only under §11.4.119, unlike the hermetic siblings which never skip). The `run-tests.sh` `test_regression_guards()` wiring should treat exit **2** as a §11.4.3 SKIP (`test_result "..." "SKIP"`), exit **0** as PASS, exit **1** as FAIL — for BOTH the GREEN and the `RED_MODE=1` invocation. The conductor makes that wiring change (and any phase3 hook change); this guard does not edit `deploy/` or `run-tests.sh`.

Edge cases

- **Image / podman-compose absent** → SKIP(2) with reason `topology_unsupported` / `feature_disabled_by_config` (mirrors phase3's own exit-2 OPERATOR-BLOCKED precondition). Never a fake pass.
- **phase3 precondition exit 2** (no free port, etc.) mid-run → SKIP(2).
- **phase3 exit 0 but no verdicts file** → FAIL (a §11.4.107 missing-evidence bluff — a PASS with no captured evidence is refused).
- **Stale verdicts** → the guard reads only the `cert_analyzer_verdicts.txt` produced **after** its own start marker (`-newer`), so a previous run's file is never mistaken for this run's (single-owner, §11.4.119).
- **Missing `ionice`/`nice`** → the resource-cap prefix is built only from the tools present, so a missing cap tool never becomes a §11.4.1 script-internal FAIL-bluff.

Internal behaviour

1. Resolve `REPO_ROOT`, the phase3 artifact, and the phase3 evidence root.
2. Source `tests/lib/evidence.sh` when present (optional §11.4.69 emit helpers).
3. Preconditions: phase3 present, `podman`/`podman-compose` on `PATH`, the built Caddy image exists — else honest §11.4.3 SKIP(2).
4. Drop a start marker, then invoke phase3 capped `GOMAXPROCS=2 nice -n 19 ionice -c 3` (GREEN: shipped defaults with `ACME_RESOLVERS` unset; RED: `ACME_RESOLVERS=challtestsrv:8053`), capturing its combined output to `qa-results/regression/phase3_issuance_guard/`.

5. GREEN — on phase3 exit 0, read the newest post-marker `cert_analyzer_verdicts.txt` and assert both `cert_chain_roots_in: PASS` and `cert_san_matches: PASS`. RED — assert phase3 FAILED (non-0, non-2).
6. Emit the PASS/FAIL/SKIP verdict + an evidence file; exit `0`/`1`/`2`.

The guard itself boots nothing and writes only its own evidence file; phase3 boots and self-tears-down the hermetic stack on every exit path (§11.4.14).

Related scripts

- `deploy/letsencrypt/phase3_hermetic_issue.sh` — the issuance artifact under guard.
- `tests/letsencrypt/cert_analyzer.sh` — produces the verdicts this guard asserts.
- `tests/regression/cert_analyzer_selfvalidation_test.sh` — sibling §11.4.135 guard for the analyzer's self-validation (Phase 1).
- `tests/regression/assert_egress_ip_host_unknown_test.sh` — sibling guard whose RED_MODE polarity + style this guard matches.
- `docs/research/letsencrypt_hermetic_20260701/` — CoreDNS-SOA-front root-cause research.

Constitution anchors

§11.4.107 (real-evidence / liveness), §11.4.108 (source→artifact→runtime), §11.4.115 (RED-baseline polarity switch), §11.4.135 (standing regression guard), §11.4.3 (SKIP-with-reason), §11.4.50 (determinism), §11.4.119 (single-resource-owner — conductor-only boot), §1.1 (paired mutation).

Last verified: 2026-07-01 (authored; `sh -n` + `bash -n` clean; not executed — boot is conductor-only per §11.4.119).